

Project Asteria - Session 2: Architecture Notes

Core Philosophy

Design before code. Build an autonomous robot, not just Python functions.

Robot State

- Left and right wheel speeds
- Position: x, y, theta (orientation)
- Front distance sensor (initially one sensor)

Motion Model

Encoders estimate wheel motion. Wheel speeds determine forward velocity. Distance traveled updates position while heading (theta) determines direction.

Architecture Evolution

Instead of one giant Robot class, separate responsibilities:

- Robot: Coordinator/orchestrator.
- Perception: Reads sensors.
- Localization: Estimates robot pose.
- Decision: Chooses next action.
- Planner (future): Converts goals into plans.
- Controller: Converts plans into motor commands.
- World Model / State Estimator (future): Maintains a unified view of the world.
- Safety Manager (future): Emergency override.

Key Principles

- Single Responsibility Principle.
- Robot coordinates modules; specialists perform work.
- Information flows upward (sensors to world model).
- Commands flow downward (decision to motors).
- One source of truth for robot state.
- Sensor fusion combines multiple imperfect measurements.
- Commands are temporary; robot state is persistent.

Behavior Loop

Perceive -> Localize -> Build/Update World Model -> Decide -> Plan -> Control -> Repeat continuously.

Important Design Insights from Riyan

- Use continuous feedback instead of move-for-fixed-time commands.
- Robot should coordinate independent specialist classes.
- Decision should receive a unified snapshot rather than querying every module directly.
- Architecture should allow modules to evolve independently.
- Future robots with different missions should reuse the same architecture by changing only high-level objectives.

Open Questions for Future Sessions

- Exact responsibility of Robot vs. World Model.
- Mission/Task Manager/Objectives module.
- Planning hierarchy.
- Emergency safety overrides.
- Implementation of each class in Python.